

Lab-3: CS-2203 Artificial Intelligence Lab

Assignment No: 03

Date: Jan 22, 2026

Submission Deadline: [Jan 22, 2026 - 17:00 Hrs. IST]

Instructions for Submission

General Instructions

Some assignments may need to be submitted via Google Form. Please carefully follow the instructions below to ensure proper submission:

File Preparation

Submission Instructions:

1. Store all assignment-related files in a single folder and compress it into a `.rar` or `.zip` file. The compressed file must be named in the following format:

`roll-number_assign3.rar` or `roll-number_assign3.zip`

2. For example, if your roll number is 2201AI01, the compressed file should be named as:

`2201AI01_assign1.rar` or `2201AI01_assign1.zip`

3. Save each program using the filename format specified alongside each question.
4. If all assignments are completed within a single Jupyter Notebook, name the notebook file using the following format:

`CS2203_Lab3.ipynb`

5. When using a single notebook for multiple problems, clearly **segregate the code for each problem**. Each problem must be introduced using a **text cell** containing the problem statement before the corresponding code segment.
6. After completing the notebook, place it inside the assignment folder, compress the folder, and upload it by following the above naming instructions.

7. Additionally, submit a **PDF containing both the code and its output**. To generate the PDF:

- Open the Jupyter Notebook file.
- Navigate to **File** → **Print**.
- Save the file as a PDF using the same name as the notebook (e.g., `CS2203_Lab3.pdf`).

Submission Link

Upload your compressed file using the following Google Form link:

<https://forms.gle/TjdHMQZWw7Vrx5ru6>

Problem Statement : 1 (Save as 1.ipynb) [10 marks]

Write a **Python program from scratch** to visualize the output of an image after applying **horizontal and vertical convolution kernels separately** using a Convolutional Neural Network (CNN) implemented in PyTorch .

Objective

- To understand the working of convolution kernels in CNNs.
- To apply horizontal and vertical edge detection filters on an image.
- To visualize the extracted feature maps.

Theory

Convolutional Neural Networks (CNNs) analyze images by applying small filters, called convolution kernels, that slide over the image and highlight important patterns. These filters help the network identify basic visual features such as edges, textures, and shapes. In particular, directional edge detection kernels are used to capture changes in brightness along specific directions, allowing the model to clearly detect horizontal and vertical boundaries in an image.

Horizontal Edge Detection Kernel

Detects horizontal edges in an image:

$$K_h = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Vertical Edge Detection Kernel

Detects vertical edges in an image:

$$K_v = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

These kernels highlight regions of rapid intensity change along horizontal and vertical directions respectively.

Problem Statement : 2 (Save as 2.ipynb) [10 marks]

Write a **Python program from scratch** to design and implement a Convolutional Neural Network (CNN) using 32 filters and (3 x 3 kernal to each) with a stack of five convolutional blocks, visualize the intermediate feature maps, and analyze the effect of ReLU, Leaky ReLU, and Tanh activation functions in the final classification layer implemented in PyTorch.

Objective

- To understand deep feature extraction using stacked CNN layers.
- To visualize feature maps after each convolutional block.
- To study the effect of different activation functions in the classification layer.
- To compare ReLU, Leaky ReLU, and Tanh based on output behavior.

Dataset Link:

<https://drive.google.com/drive/folders/1Ab5GfR-rD1utF1r8Q5rdGxjUstu5Mgb9?usp=sharing>

Activation Functions

ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

ReLU is computationally efficient and helps reduce the vanishing gradient problem.

Leaky ReLU:

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

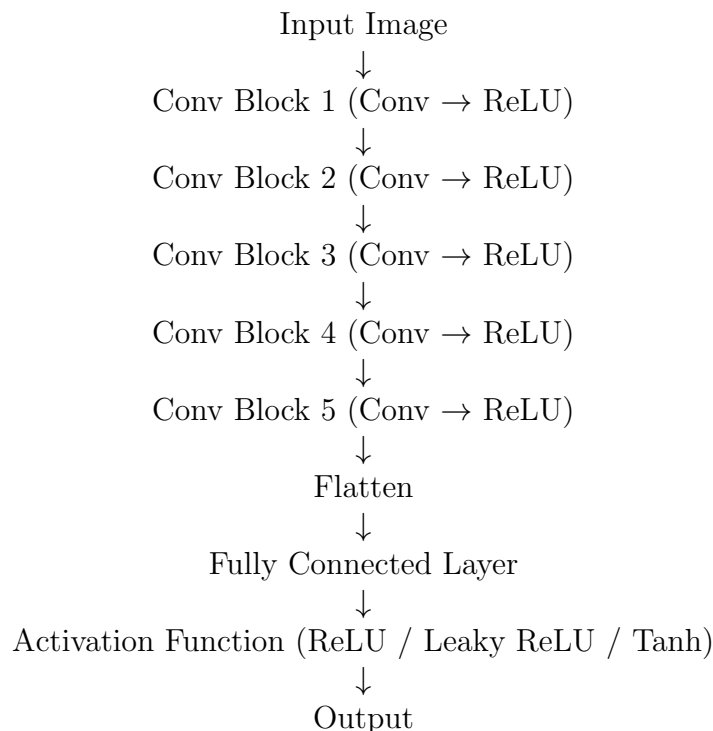
Leaky ReLU allows a small gradient for negative inputs, preventing dead neurons.

Tanh:

$$f(x) = \tanh(x)$$

Tanh outputs values in the range $(-1, 1)$ and is zero-centered but may suffer from vanishing gradients.

Architecture Description



Evaluation Criteria : 1

The evaluation of this lab assignment will be based on the following criteria:

1. **Correctness of Implementation (40%):**
2. **Code Quality & Q&A (30%):**
 - The code should be well-structured, properly commented, and follow standard Python programming practices.
 - Students may be asked questions related to their implementation to assess their understanding.
3. **Error Handling (10%):**
 - The code gracefully handles potential errors, such as file not found or invalid image formats.
4. **Documentation (10%):**
 - The code is accompanied by appropriate documentation and comments that explain the steps taken in each part of the program.
5. **Timeliness of Submission (10%):**
 - The assignment is submitted on time, adhering to the submission deadline.

Evaluation Criteria: 2

The evaluation of this lab assignment will be based on the following criterion:

1. **Viva (10 Marks):**
 - Questions will be asked related to the assignment to assess the student's understanding of the implemented concepts.